EM

EMPLOYEE MANAGEMENT SYSTEM

Technical Documentation

Full-Stack Web Application with Role-Based Access Control and Security Audit Logging

---

Author	Nguyen Hoang Viet
Project Type	Full-Stack / Security-Focused Web Application
Technology	Node.js, Express.js, EJS, MySQL
Live Demo	https://employee-management-od9a.onrender.com
Version	1.0 - October 2026

Prepared for project presentation, portfolio, and technical review.

Table of Contents

1. Introduction 3
2. Project Objectives and Scope 4
3. System Requirements 4
4. Technology Stack 5
5. System Architecture 6
6. Database Design 6
7. Authentication and Session Management 7
8. Role-Based Access Control 9
9. Employee Management 10
10. Admin Dashboard 14
11. User Management 15
12. Audit Logging 16
13. Security Controls 17
14. Deployment 18
15. Testing and Verification 19
16. Challenges and Solutions 20
17. Future Improvements 21
18. Conclusion 21
Appendix A. Main Application Routes 21
Appendix B. Environment Variables 22

Screenshots are already inserted in this document. When replacing an image, keep a similar size to minimize changes to page layout and page numbering.

1. Introduction

1.1 Project Overview

The Employee Management System is a full-stack web application developed to manage employees, application users, access permissions, and administrative activity through a centralized interface. The application is built with Node.js, Express.js, EJS, and MySQL and is deployed as a public web application.

The system supports three roles: Employee, Manager, and Admin. Each role is assigned a different set of permissions. In addition to standard employee CRUD operations, the application provides an administrative dashboard, user role management, persistent sessions, security audit logging, HTTP security headers, rate limiting, and a Content Security Policy.

1.2 Project Motivation

A basic employee management application normally focuses on storing and editing employee records. A more realistic system must also control who can perform sensitive operations, preserve authenticated sessions, record important activity, and provide administrators with useful visibility into system behavior.

This project therefore extends a traditional CRUD application into a security-focused management platform. It combines full-stack web development concepts with access control, security monitoring, and production deployment practices.

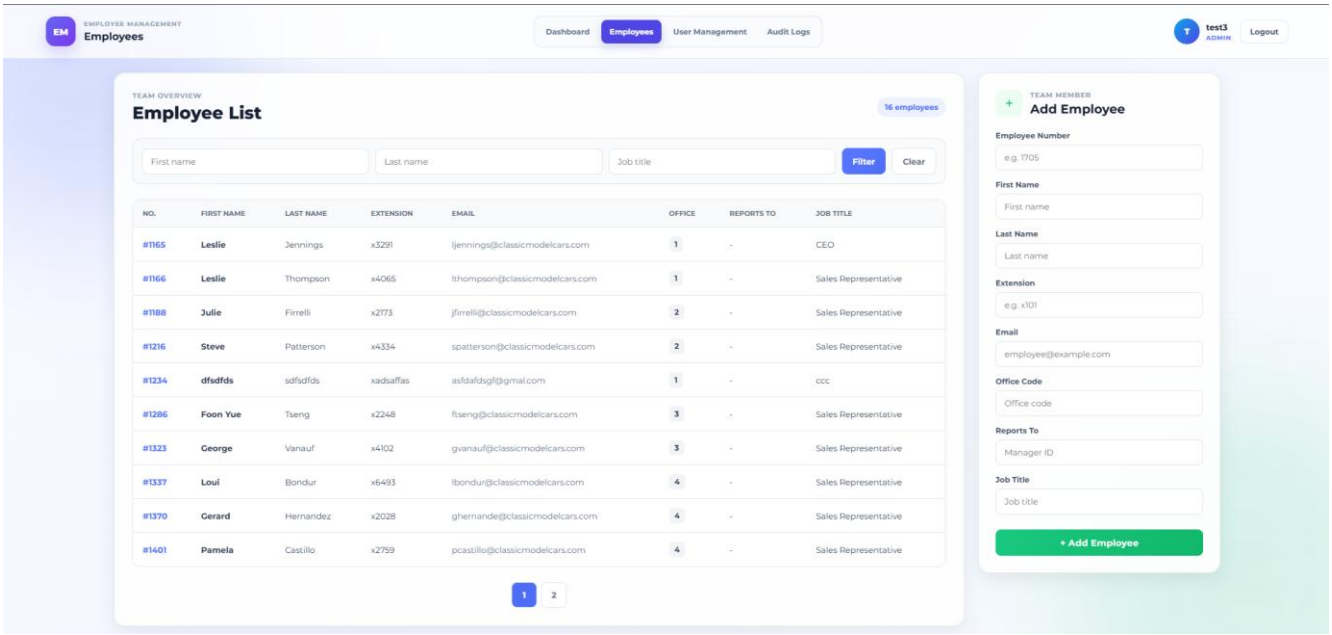


Figure 1. Employee Management System - Admin Dashboard overview

2. Project Objectives and Scope

2.1 Objectives

- Build a web-based employee management system using Node.js, Express.js, EJS, and MySQL.
- Implement user registration, login, and logout with hashed passwords.
- Apply Role-Based Access Control (RBAC) for Employee, Manager, and Admin roles.
- Allow managers to create and update employee information.
- Allow administrators to delete employees and manage application users.
- Provide an administrative dashboard using real database statistics.
- Record authentication and administrative events in an audit log.
- Store sessions persistently in MySQL.
- Add security controls such as Helmet, Content Security Policy, and rate limiting.
- Deploy the application using Render with an Aiven MySQL database.

2.2 Project Scope

The project focuses on employee data management, application user management, access control, security event visibility, and deployment. It is intended as an educational and portfolio project rather than a complete enterprise HR platform.

Scope note Payroll, attendance, performance reviews, HR workflow approvals, and enterprise identity federation are outside the current project scope.

3. System Requirements

3.1 Functional Requirements

ID	Requirement
FR01	Users can register an account.
FR02	Users can log in and log out.
FR03	Authenticated users can view employee information.
FR04	Users can search and filter employees.
FR05	Managers can add new employees.
FR06	Managers can update employee information.
FR07	Administrators can delete employees.
FR08	Administrators can access the Admin Dashboard.
FR09	Administrators can view registered application users.
FR10	Administrators can change user roles.
FR11	Administrators can review audit logs.

ID	Requirement
FR12	The system records authentication and administrative events.

3.2 Non-Functional Requirements

- Security: passwords must not be stored as plain text and protected routes must enforce authorization.
- Usability: the interface should remain consistent across the main administrative pages.
- Persistence: application data and sessions should survive normal application restarts.
- Maintainability: routes, controllers, middleware, models, views, and static assets should be separated by responsibility.
- Observability: important security and administrative events should be recorded for review.
- Deployability: the application should run in a production hosting environment using environment variables for sensitive configuration.

4. Technology Stack

Layer	Technology	Purpose
Backend	Node.js	Server-side JavaScript runtime.
Web Framework	Express.js	Routing, middleware, HTTP request handling, and application flow.
View Engine	EJS	Server-side rendering of dynamic HTML pages.
Frontend	HTML5 / CSS3 / JavaScript	Interface structure, styling, and client-side behavior.
Charts	Chart.js	Visualizes dashboard statistics.
Database	MySQL	Stores employee, user, session, and audit data.
Passwords	bcrypt	Hashes and verifies user passwords.
Sessions	express-session + express-mysql-session	Maintains authenticated sessions in MySQL.
Security Headers	Helmet	Configures security-related HTTP headers and CSP.
Rate Limiting	express-rate-limit	Limits repeated requests to authentication endpoints.
Hosting	Render	Hosts the Node.js web application.
Managed Database	Aiven MySQL	Hosts the production MySQL database.

5. System Architecture

5.1 Architectural Style

The application follows an MVC-style separation of responsibilities. Express routes map incoming HTTP requests to controller functions. Middleware performs authentication, authorization, and

request protection. Controllers coordinate application logic and call model/query functions. Models communicate with MySQL, while EJS views render HTML responses.

Layer	Main Responsibility
Routes	Map URLs and HTTP methods to controller actions.
Middleware	Authentication, role checks, rate limiting, and request processing.
Controllers	Coordinate request handling and application logic.
Models / Queries	Execute SQL queries against MySQL.
Views	Render server-side EJS pages.
Public Assets	Serve CSS and client-side JavaScript.

5.2 Typical Request Flow

1. A user sends an HTTP request from the browser.

2. Express matches the request to a route.

3. Authentication and role middleware determine whether the user is allowed to continue.

4. The controller validates the request and invokes the required model/query function.

5. The model communicates with MySQL.

6. The controller renders an EJS view or redirects the user.

7. The browser receives the generated HTML response.

6. Database Design

6.1 Main Data Entities

The application uses MySQL as the primary relational database. The most important entities for the application layer are employees, users, audit logs, and sessions.

Table	Purpose
employees	Stores employee profile and organizational information.
users	Stores application accounts, password hashes, employee links, and roles.
audit_logs	Stores authentication and administrative security events.
sessions	Stores persistent Express session data.

6.2 Audit Log Structure

Column	Description
id	Unique audit event identifier.
user_id	Application user ID when available.

Column	Description
username	Username associated with the event.
action	Event type such as LOGIN_FAILED or EMPLOYEE_UPDATED.
target	Object affected by the action.
ip_address	Source IP address recorded by the application.
created_at	Event creation timestamp.

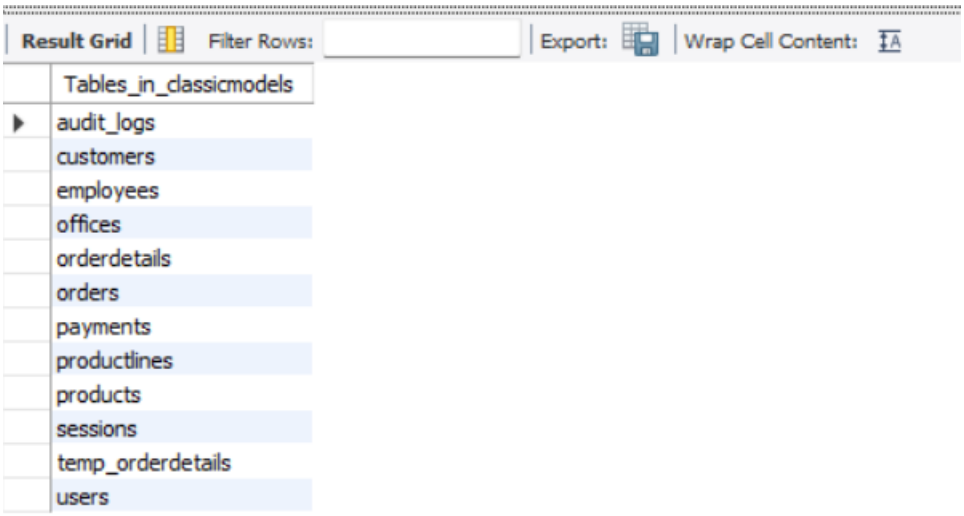


Figure 2. Database schema / main application tables

7. Authentication and Session Management

7.1 Registration and Login

Users authenticate with an application account. Passwords are processed with bcrypt so the database stores password hashes rather than plain-text passwords. During login, the submitted password is compared with the stored hash before an authenticated session is created.

7.2 Session Storage

The application uses express-session together with express-mysql-session. Session data is stored in MySQL rather than only in application memory, improving persistence across normal server restarts and making the design more suitable for deployment.

7.3 Session Cookie Configuration

- httpOnly prevents normal client-side JavaScript from reading the session cookie.
- sameSite=lax reduces some cross-site request contexts while preserving common navigation behavior.
- secure is enabled in production so the session cookie is transmitted over HTTPS.
- A maximum session lifetime is configured to limit session duration.

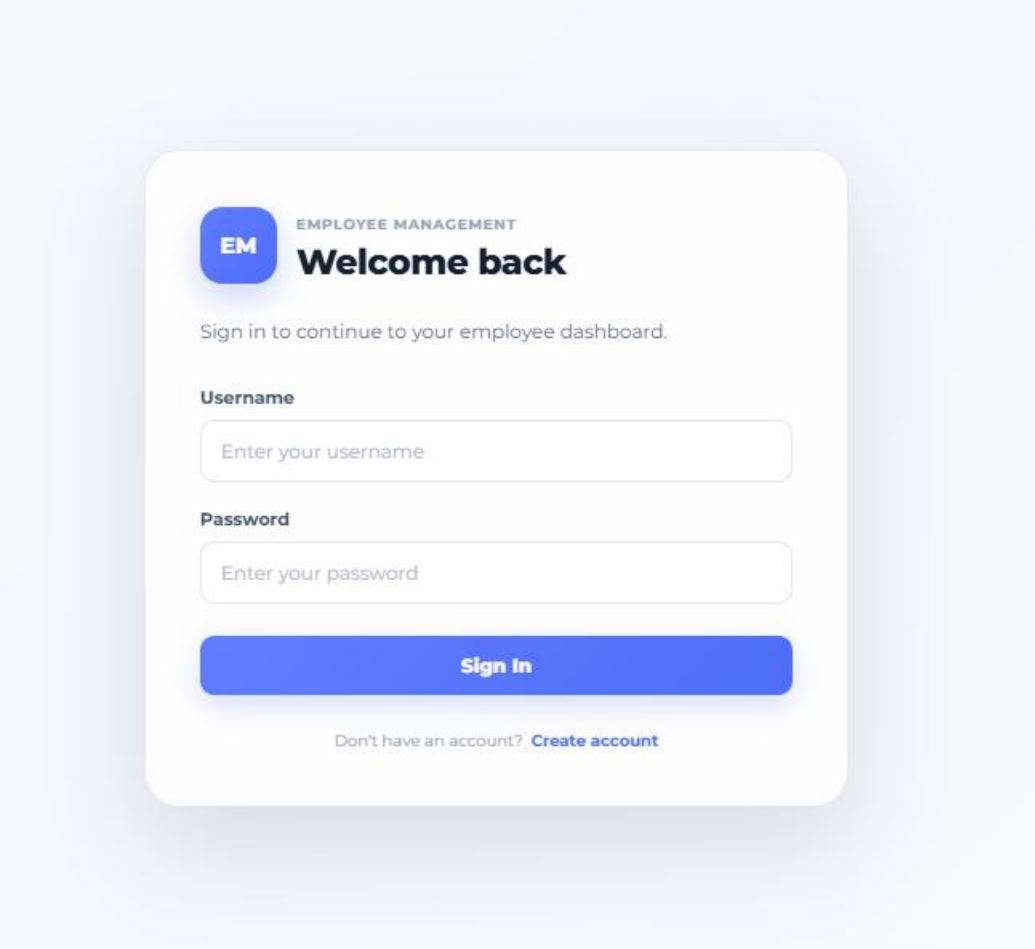


Figure 3. User login interface

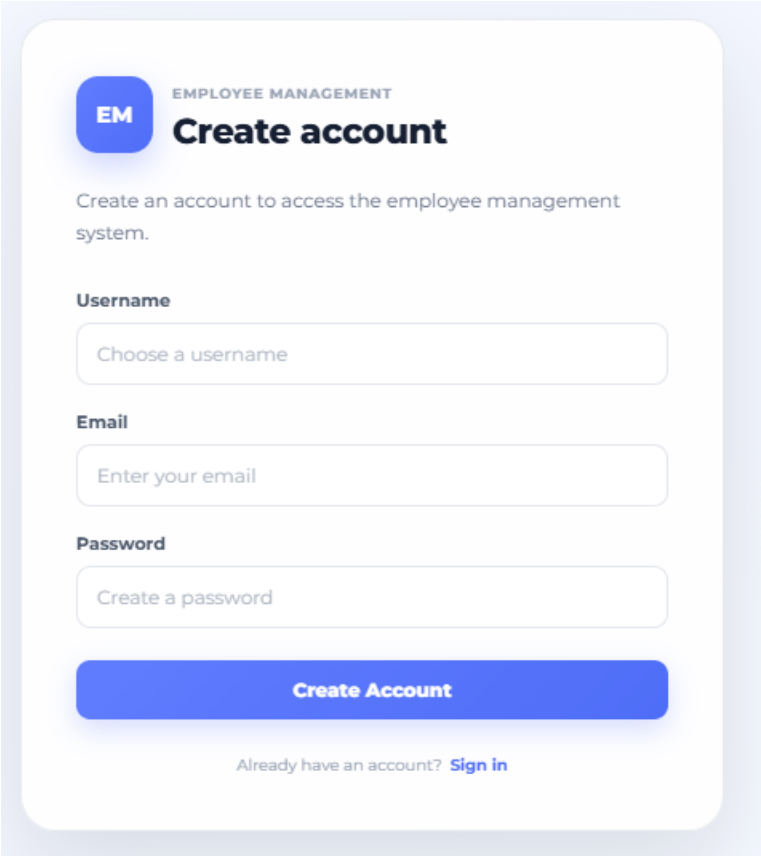


Figure 4. User registration interface

8. Role-Based Access Control

8.1 Roles

Authorization is based on three application roles. The role attached to the authenticated session determines which routes and interface actions are available to the user.

Feature	Employee	Manager	Admin
View employees	Yes	Yes	Yes
Search / Filter employees	Yes	Yes	Yes
Add employee	No	Yes	Yes
Edit employee	No	Yes	Yes
Delete employee	No	No	Yes
Admin Dashboard	No	No	Yes
User Management	No	No	Yes
Change user roles	No	No	Yes
View Audit Logs	No	No	Yes

8.2 Enforcement

Authorization is enforced on the server through authentication and role middleware, not only by hiding buttons in the interface. This is important because client-side visibility alone is not a security control.

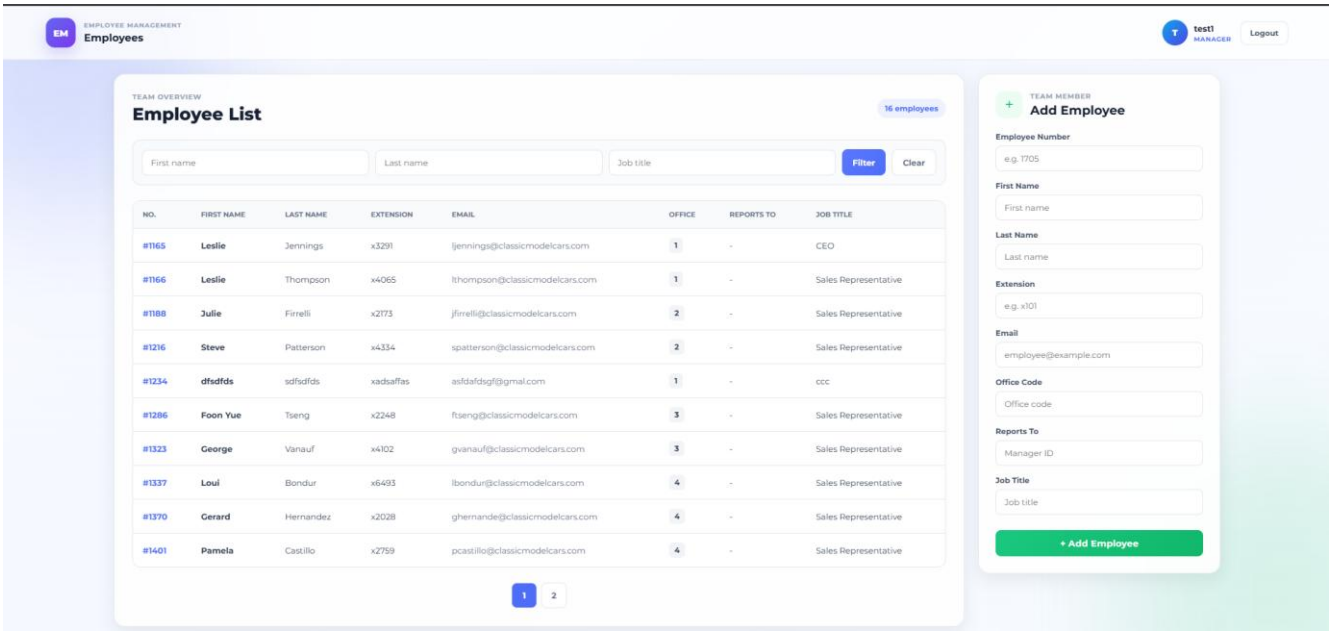


Figure 5. Role-based interface differences

9. Employee Management

9.1 Employee List and Filtering

Authenticated users can view employee data in a tabular interface. Search/filter controls allow the user to narrow results by employee attributes, while pagination keeps larger result sets manageable.

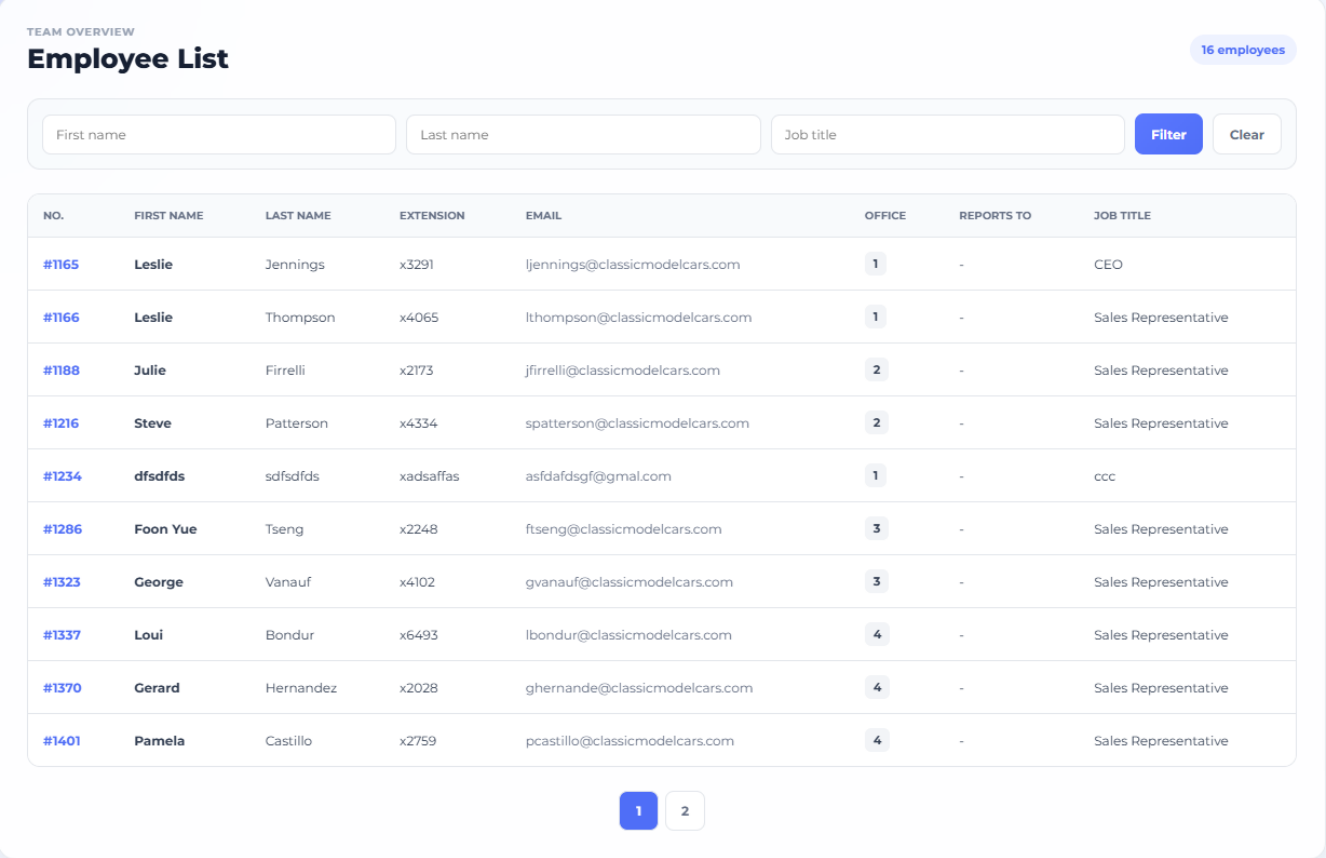


Figure 6. Employee list and filtering interface

9.2 Add Employee

Managers and administrators can create new employee records using the Add Employee form. Required fields are submitted to the server and inserted into MySQL through the model/query layer.

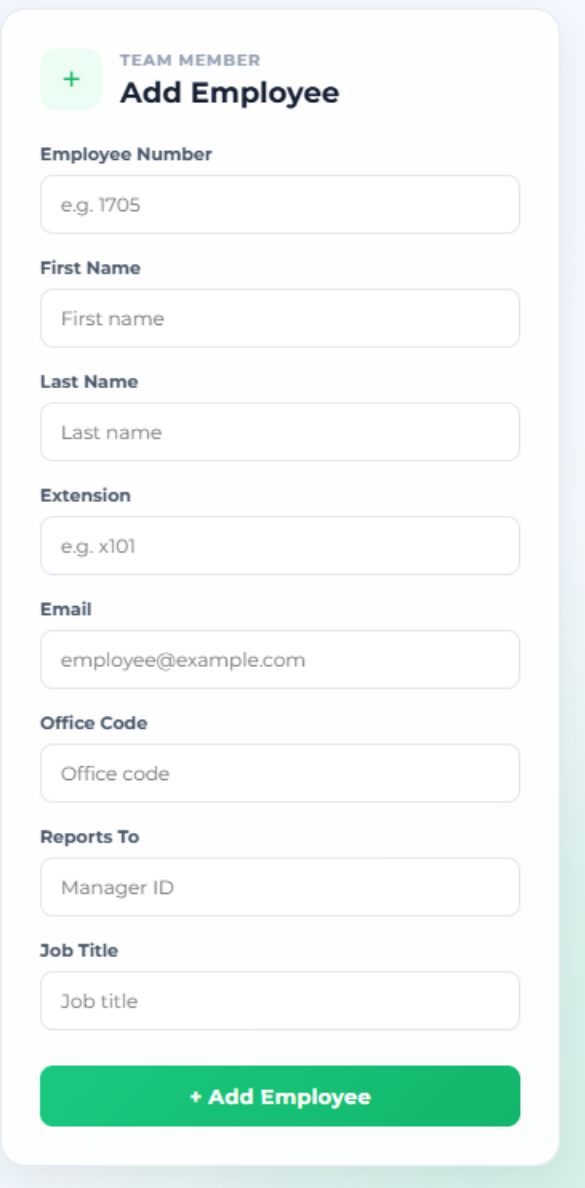


Figure 7. Add Employee form

9.3 Edit Employee

Managers and administrators can open an employee record and update its information. The edit interface loads the selected employee data into a modal form before the changes are submitted to the server.

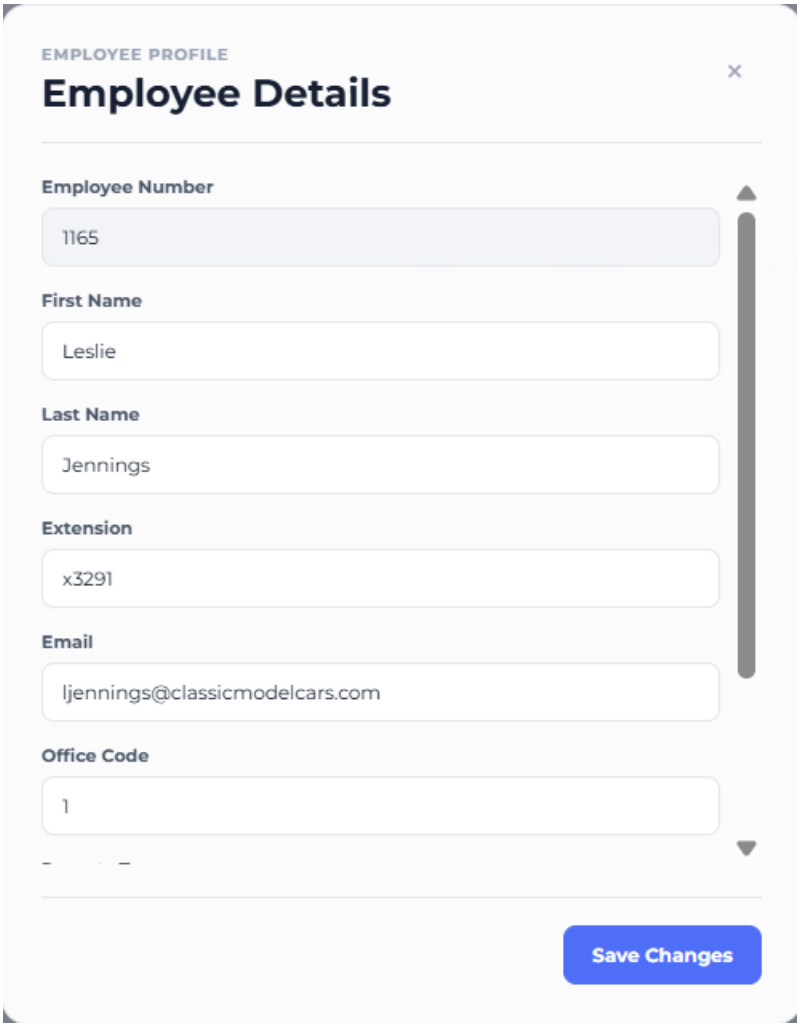


Figure 8. Edit Employee modal

9.4 Delete Employee

Employee deletion is restricted to administrators. A confirmation dialog is displayed before the destructive action is executed, reducing accidental deletion.

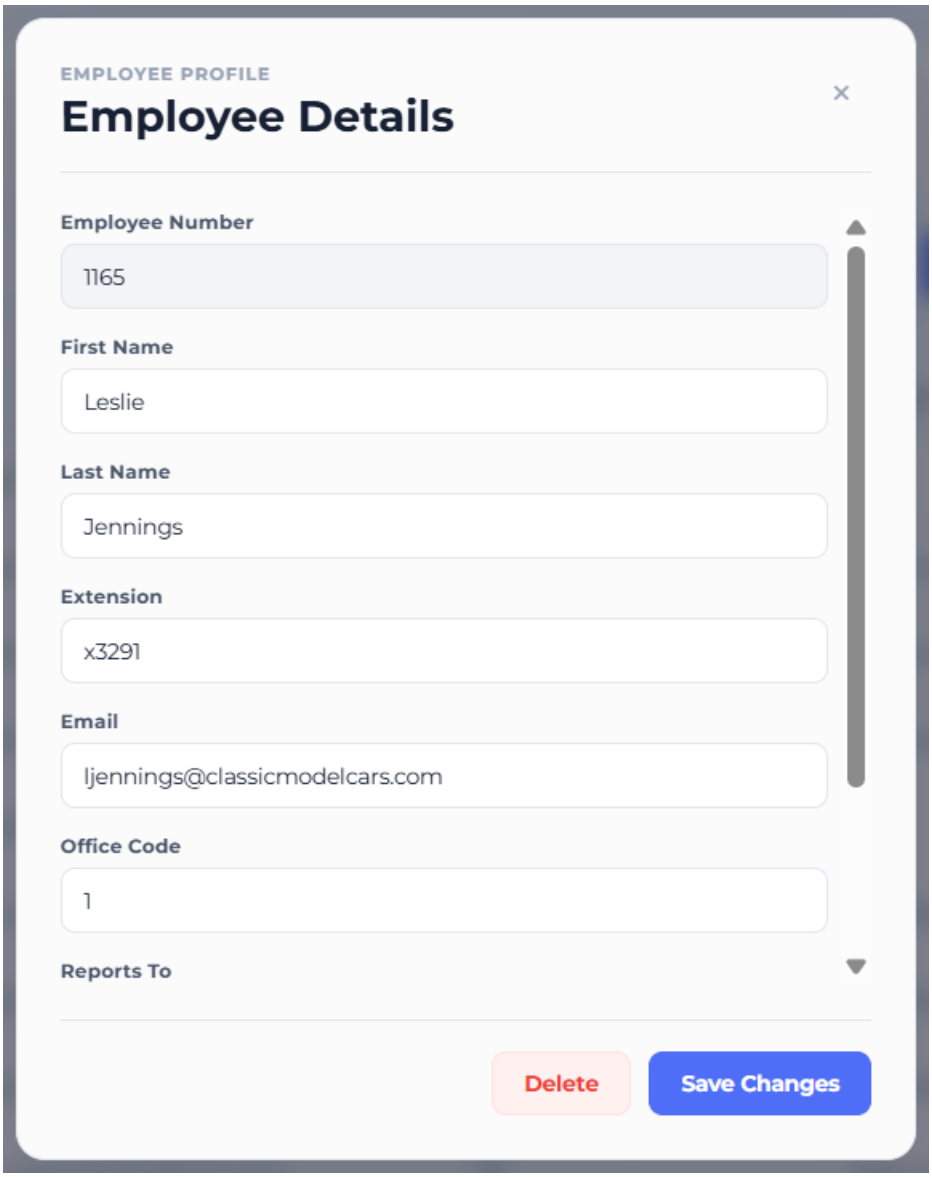


Figure 9. Delete Employee confirmation

10. Admin Dashboard

10.1 Dashboard Purpose

The Admin Dashboard provides a centralized overview of employee, user, and security activity using data queried from MySQL. It is restricted to administrators.

10.2 Summary Metrics

- Total Employees
- Total Users
- Manager count
- Administrator count
- Failed Login count

- Security Events Today

10.3 Charts

- Users by Role - doughnut chart.
- Employees by Office - bar chart using office names.
- Audit Activity - line chart covering the last seven days.
- Audit Events by Action - bar chart grouped by audit event type.

Dashboard JavaScript is stored in a separate static file rather than inline script blocks. This supports a stricter Content Security Policy by avoiding the need for script-src unsafe-inline.

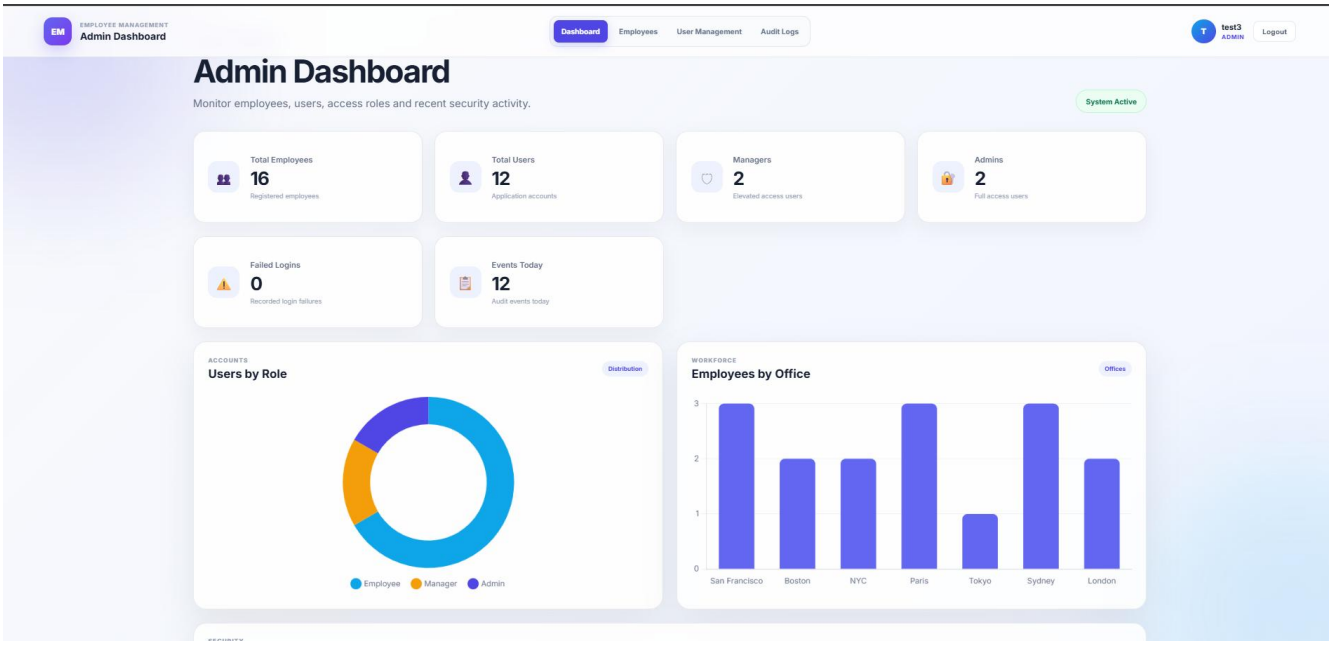


Figure 10. Admin Dashboard with live statistics and charts

11. User Management

11.1 User Administration

The User Management page allows administrators to review registered accounts, inspect linked employee IDs, view current roles, and change a user role when necessary.

The currently authenticated administrator is protected from accidentally changing their own role through the normal role management interface.

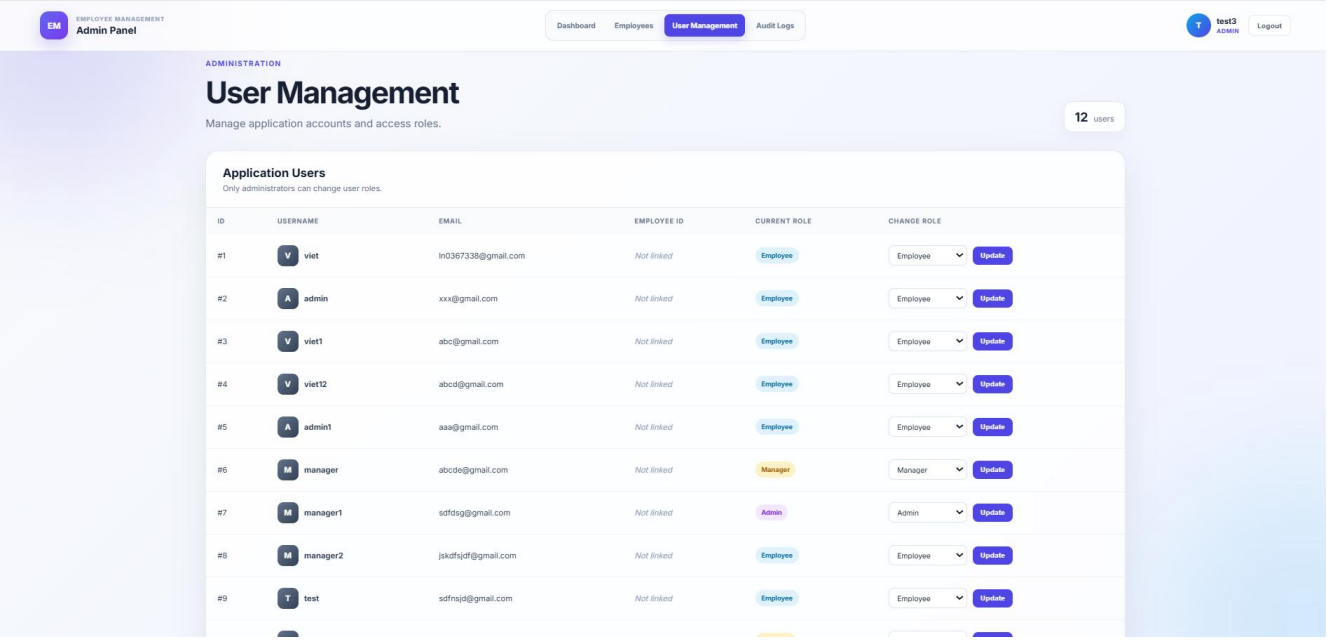


Figure 11. Administrator User Management interface

11.2 Role Change Auditability

When an administrator changes a user role, the application writes a USER_ROLE_UPDATED event to the audit log. The audit target identifies the affected user and records the old and new role values.

12. Audit Logging

12.1 Purpose

Audit logging provides visibility into important security and administrative actions. It allows an administrator to review what happened, which user was involved, the target of the action, the source IP address, and when the event occurred.

12.2 Recorded Event Types

Event	Meaning
LOGIN_SUCCESS	Successful authentication.
LOGIN_FAILED	Failed authentication attempt.
LOGOUT	Authenticated user logged out.
EMPLOYEE_CREATED	Employee record was created.
EMPLOYEE_UPDATED	Employee record was modified.
EMPLOYEE_DELETED	Employee record was deleted.
USER_ROLE_UPDATED	Application user role was changed.

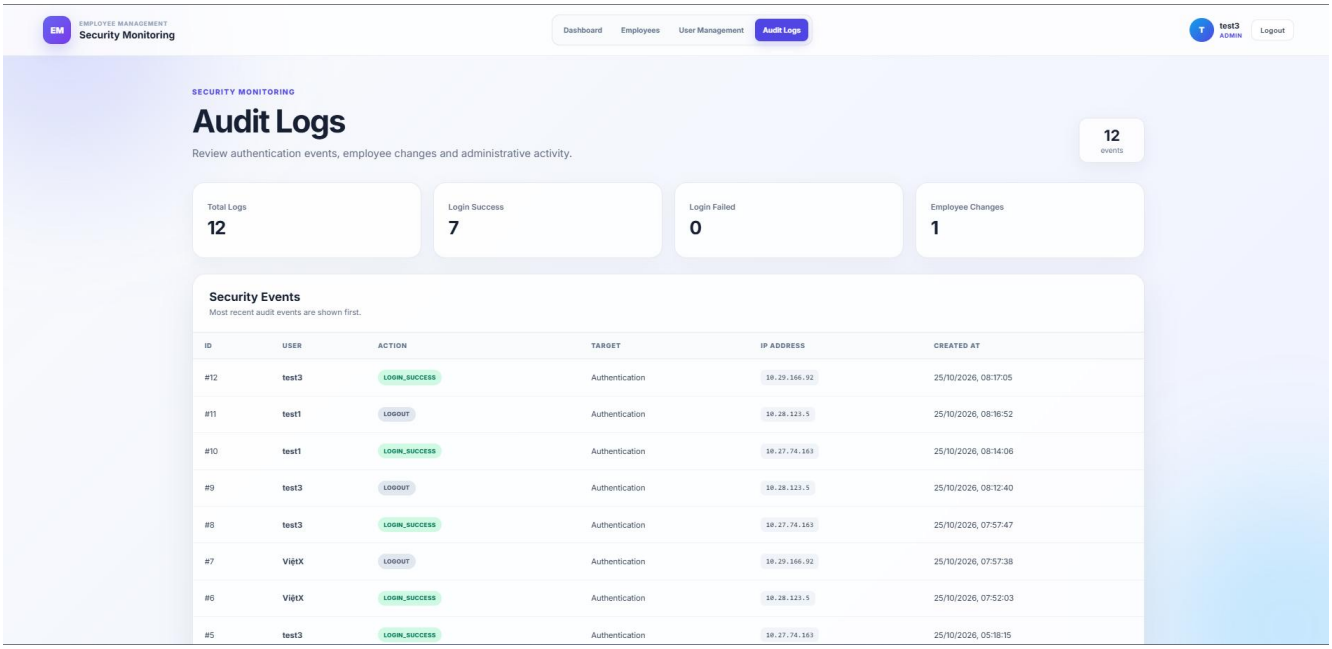


Figure 12. Security and administrative audit events

13. Security Controls

13.1 Password Protection

bcrypt is used to hash passwords before storage. Login verification compares the submitted password against the stored password hash.

13.2 HTTP Security Headers

Helmet is used to configure security-related HTTP headers. The application also defines a Content Security Policy that restricts the sources used for scripts, styles, fonts, images, and network connections.

13.3 Content Security Policy

The current policy allows application scripts from the same origin and Chart.js from jsDelivr. Dashboard code was moved to an external static JavaScript file so script-src does not require unsafe-inline. The application still permits inline styles where required by the current frontend design.

13.4 Rate Limiting

Login and registration endpoints use express-rate-limit. The login limiter reduces repeated authentication attempts within a time window, helping limit automated brute-force behavior. Successful logins are skipped by the failed-attempt-oriented login limiter configuration.

13.5 Server-Side Authorization

Sensitive routes are protected through authentication and role middleware. This prevents unauthorized actions even if a user manually attempts to access a protected URL.

13.6 Security Event Monitoring

Failed and successful login events, employee changes, logout events, and user role modifications are stored in the audit log and surfaced in the administrative interface and dashboard.

> **Current limitation** CSRF protection is not part of the current stable release. It is listed as a future improvement so it can be introduced together with complete form-token integration and regression testing.

14. Deployment

14.1 Production Environment

The web application is deployed on Render. The production MySQL database is hosted on Aiven. Sensitive configuration such as database credentials and session secrets is provided through environment variables rather than hard-coded in source files.

Live Demo:

https://employee-management-od9a.onrender.com

14.2 Deployment Flow

1. Source code is pushed to the Git repository.
2. Render builds and starts the Node.js application.
3. Production environment variables provide database and session configuration.
4. The application connects to Aiven MySQL.
5. Users access the application over HTTPS through the deployed Render URL.

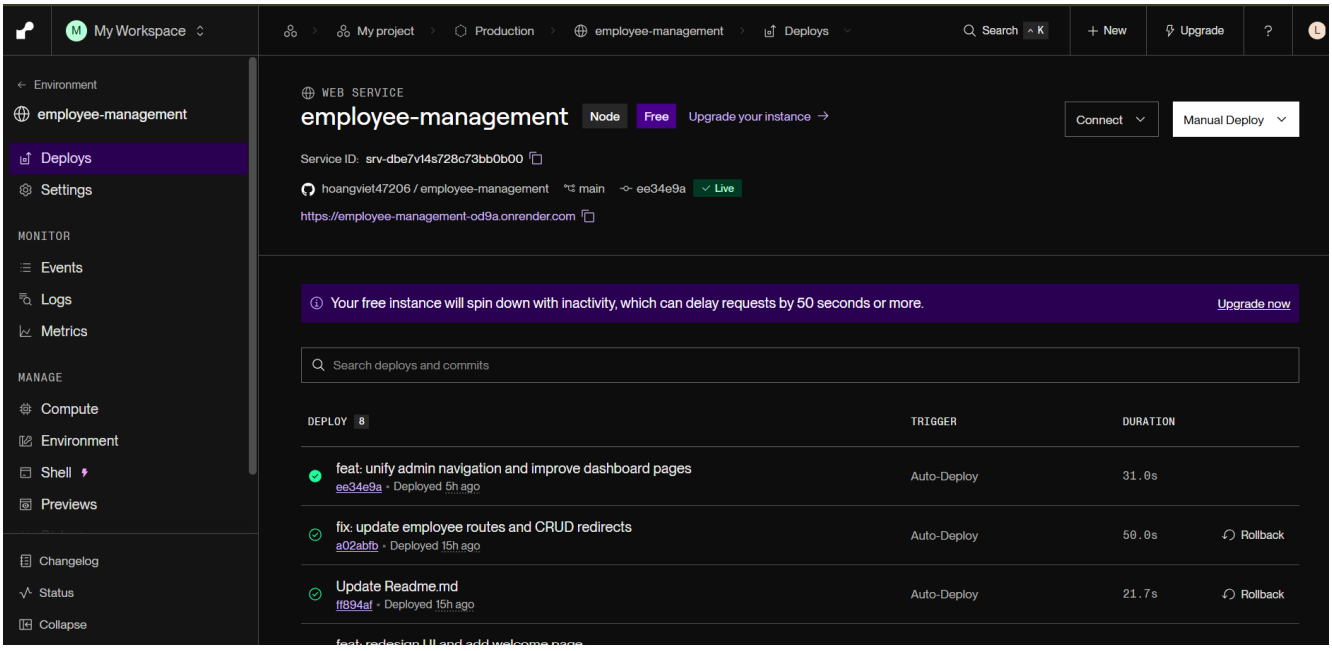


Figure 13. Production deployment on Render

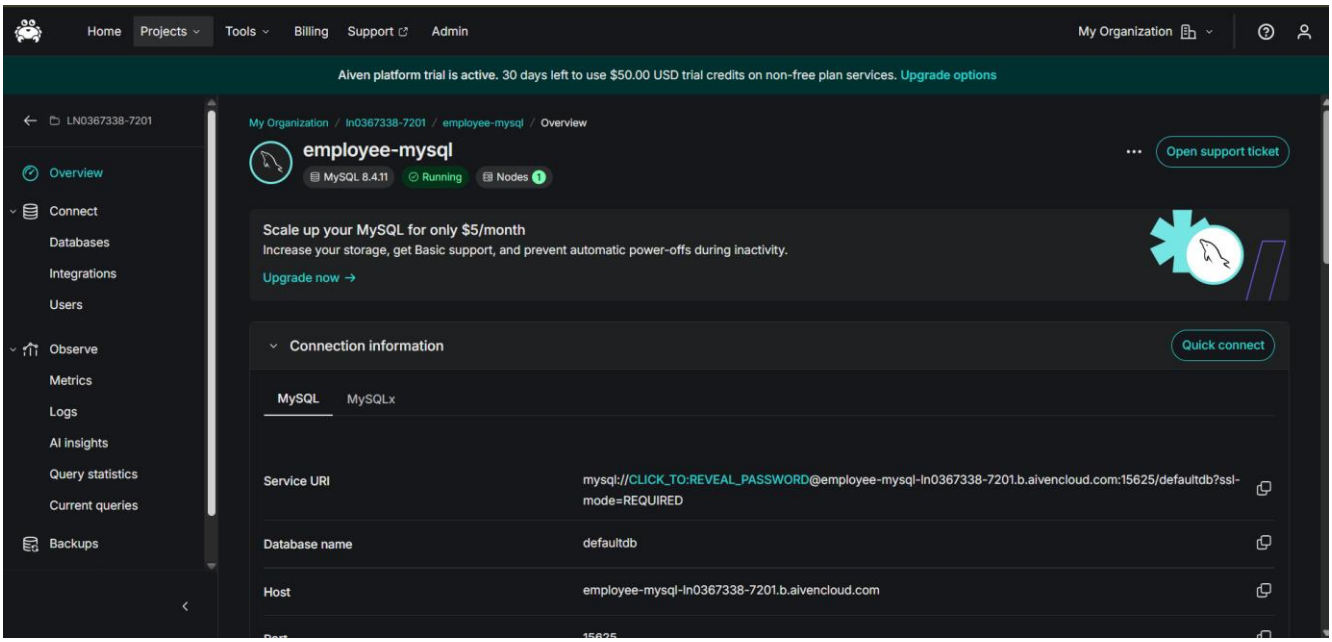


Figure 14. Managed MySQL database on Aiven

15. Testing and Verification

15.1 Functional Test Cases

Test	Expected Result	Status
Valid login	Authenticated session is created and the user is redirected.	Pass
Invalid password	Login is rejected and LOGIN_FAILED is recorded.	Pass
Employee views employees	Employee list is accessible.	Pass
Employee accesses admin page	Access is denied.	Pass
Manager adds employee	Employee is created and audit event is recorded.	Pass
Manager edits employee	Employee is updated and audit event is recorded.	Pass
Manager deletes employee	Operation is not permitted.	Pass
Admin deletes employee	Employee is deleted after confirmation.	Pass
Admin changes user role	Role is updated and USER_ROLE_UPDATED is recorded.	Pass
Admin opens dashboard	Dashboard metrics and charts are displayed.	Pass
Logout	Session is terminated and LOGOUT is recorded.	Pass

15.2 Security-Oriented Verification

- Verify that a non-admin user cannot access /admin/dashboard, /admin/users, or /audit-logs.
- Verify that manager and employee roles cannot perform admin-only deletion.
- Verify that failed logins appear in the audit log.
- Verify that repeated login requests eventually trigger rate limiting.

- Verify that session cookies are configured securely in production.
- Verify that dashboard scripts operate under the configured Content Security Policy.

16. Challenges and Solutions

Challenge	Solution
Persistent authentication after deployment	Moved session storage from in-memory sessions to MySQL using express-mysql-session.
Need for role-specific permissions	Introduced authentication and role middleware and enforced authorization on protected routes.
Need to understand administrative activity	Added an audit_logs table and event logging for authentication, employee changes, and role changes.
Dashboard required real data	Added aggregate model queries and connected them to Chart.js visualizations.
Office chart initially showed codes rather than readable names	Joined employee office codes with office data so chart labels can display office names.
Content Security Policy conflicted with inline dashboard JavaScript	Moved chart logic to a separate public/scripts/admin-dashboard.js file and tightened script-src.
Admin pages developed with different navigation styles	Created a shared admin-navigation.css and standardized the navigation across Dashboard, Employees, User Management, and Audit Logs.
CSRF integration affected existing CRUD forms during development	Returned to the stable form flow and kept CSRF protection as a planned improvement pending complete integration and testing.

17. Future Improvements

- Complete CSRF protection for all state-changing forms.
- Password reset flow with secure time-limited tokens.
- Email verification for newly registered accounts.
- Advanced audit-log search, date filtering, and event filtering.
- Audit-log export for reporting or analysis.
- Employee profile pages with richer information.
- Additional dashboard analytics and trend comparisons.
- Automated unit, integration, and authorization tests.
- Docker Compose configuration for reproducible local deployment.
- CI/CD pipeline for automated testing and deployment.
- Account lockout or adaptive controls for repeated failed authentication.
- Stronger password policy and password-change functionality.

18. Conclusion

The Employee Management System demonstrates how a standard database-backed CRUD application can be extended with authentication, authorization, persistent sessions, security

monitoring, administrative analytics, and production deployment. The project combines web development and security concepts in a practical application.

The final system supports three permission levels, provides administrative control over users and employees, records important activity for later review, and includes multiple defensive controls. It also provides a foundation for future work such as CSRF protection, automated testing, stronger account security, and expanded monitoring capabilities.

Appendix A. Main Application Routes

Route	Purpose	Access
/	Home / welcome page	Public
/login	Login interface	Public
/register	Registration interface	Public
/employees	Employee management	Authenticated
/admin/dashboard	Administrative dashboard	Admin
/admin/users	User and role management	Admin
/audit-logs	Audit log review	Admin

Appendix B. Environment Variables

Variable	Purpose
DB_HOST	MySQL database host.
DB_PORT	MySQL database port.
DB_USER	MySQL username.
DB_PASSWORD	MySQL password.
DB_NAME	Application database name.
SESSION_SECRET	Secret used to sign session data.
NODE_ENV	Runtime environment such as development or production.
PORT	Application HTTP port.

Security reminder Never include real passwords, connection strings, session secrets, or production credentials in screenshots, documentation, README files, or the Git repository.

When inserting a screenshot, delete the corresponding gray placeholder box and insert the image in its place. Keep the caption directly below the image. Use consistent image widths (approximately 5.8-6.4 inches) and crop unnecessary browser chrome when possible.